

Nestoc 文档插件

xiebyl

一个模块+嵌套的 Typst 文档插件。

目录

1. Nestoc 简介	1
2. 使用	2
3. API	2
3.1. <code>nestoc(nestoc_fn, heading_offset:0, template0:nestemp.template0,</code> <code>templateN:nestemp.templateN) => body</code>	2
3.2. <code>nestoc_fn(heading_offset:0) => nestemp_args</code>	3
3.3. <code>template0(title:"", title-label:none, author:"", abstract:[], body) =></code> <code>styled_body</code>	3
3.4. <code>templateN(heading_offset, title:"", author:"", abstract:[], body) =></code> <code>styled_body</code>	3
4. Nestemp 模板	3
4.1. 参考文献管理	3
4.2. 秘密等级管理	3
4.3. 术语管理	4
5. 例子	4
5.1. 父模块的题目	5
5.1.1. 父模块的一级标题	5
5.1.2. 子模块的题目	5
5.1.2.1. 子模块的一级标题	5
5.1.2.2. 孙模块的题目	5
5.1.2.2.1. 孙模块的一级标题	5
6. 其他: 文档注释 (Doccom)	6
6.1. 要解决的问题	6
6.2. 实现方法	6
6.3. 比较	6
6.4. 例子	6
术语索引	7
图索引	8
表格索引	8
代码块索引	8
参考文献	8

1. Nestoc 简介

 Nestoc  的名称来源于 Nest (嵌套 ) 与 Doc (文档 ) 的组合, 亦可理解为 Nest + ToC (Table of Contents 目录)。Nestoc 旨在构建一个支持文档嵌套 (或称"模块化") 的 Typst 插件。

设想一个包含多个章节的大型文档项目，需要由多位协作者分**模块**完成各章节内容，最终将这些模块**嵌套**整合为统一的完整文档。

- **模块** 每个章节均为独立、可编辑、可编译、可阅读的自包含文档。
- **嵌套** 所有章节组合时，标题、编号等元素将自动调整，形成协调一致的最终文档。

2. 使用

每个 Nestoc 文档：

- 必须提供函数 `nestoc_fn` 供其他 Nestoc 文档调用
- 可调用 `nestoc(nestoc_fn)` 来渲染当前文档

因此一个 Nestoc 文档的代码框架如下：

```
1 // 导入 nestoc
2 #import "@local/nestoc:0.1.0"
3 // 定义 nestoc_fn 函数
4 #let nestoc_fn(heading_offset: 0) = (
5   title: "题目", author: "作者", abstract: [摘要], body: [
6     一级标题
7   ]
8   正文...
9 )
10 // 调用 nestoc 函数，将 nestoc_fn 的文档内容插入于此。
11 #nestoc.nestoc(nestoc_fn)
```

代码 1 Nestoc 文档的代码框架

若用图表示上面的 Nestoc 文档代码框架，则如下：

`nestoc_fn` — `nestoc(..)+编译` → 文档.pdf

图 1 Nestoc 文档代码框架的图示

- 第 1-9 行定义函数 `nestoc_fn(heading_offset: 0) => nestemp_args`。返回的 `nestemp_args` 的字典包含了 `title`, `author`, `abstract` 以及文档主体 `body`。
- 第 10-11 行将上面定义的 `nestoc_fn` 传给 `nestoc` 函数。`nestoc` 内部会进行标题级别的协调，然后调用 `nestoc_fn` 函数，最后返回处理好的文档主体。

你可能会好奇若不调用 `nestoc(..)` 函数会怎样？则该 `typst` 文件仅是一个无法编译出 pdf 的模块。这个模块仅能用于嵌入其他模块。

TODO:说明 nix

3. API

xieby1

Nestoc 的所有功能围绕着函数 `#nestoc(nestoc_fn, heading_offset, template0, templateN)` `=> body` 展开。

3.1. `nestoc(nestoc_fn, heading_offset:0, template0:nestemp.template0, templateN:nestemp.templateN) => body`

对 `nestoc_fn` 包含的文档内容施加 `heading_offset` 的偏移。然后根据 `heading_offset` 施加样式：若 `heading_offset==0` 则应用样式 `template0`，否则应用样式 `templateN`。

- **参数** `nestoc_fn` 为函数，见小节 3.2
- **参数** `heading_offset` 为 `int`，表示给 `nestoc_fn` 的题目和标题施加 `heading_offset` 的偏移。

例如：

- **heading_offset:0** 表示不偏移
- **heading_offset:1** 表示 nestoc_fn 题目=>一级标题, nestoc_fn 的一级标题=>二级标题, ...
- **heading_offset:2** 表示 nestoc_fn 题目=>二级标题, nestoc_fn 的一级标题=>三级标题, ...
- 参数 **template0** 为函数, 见小节 3.3
- 参数 **templateN** 为函数, 见小节 3.4
- 返回值 为文档内容

3.2. nestoc_fn(heading_offset:0) => nestemp_args

- 参数 **heading_offset** 同 nestoc 的 heading_offset
- 返回值 **nestemp_args** dict 类型, 有如下成员变量:
 - **title** content 类型, 文档题目
 - **title-label** label 类型, 文档题目的 label
 - 为什么需要 title-label, 而不直接 title: [content <label>]? 因为后者会报错"occurs multiple times in the document", 报错原因: heading 在 outline 和 body 中各出现了一次。此外后者的没有给 title/heading 打上 label, 而是给 content 打上的 label。
 - **author** str 类型, 作者
 - **abstract** content 类型, 摘要
 - **body** content 类型, 文档内容

3.3. template0(title:"", title-label:none, author:"", abstract:[], body) => styled_body

- 参数 同 nestemp_args
- 返回值 应用了模板之后的文档内容

3.4. templateN(heading_offset, title:"", author:"", abstract:[], body) => styled_body

- 比 templateN 多了参数 heading_offset。其他参数含义一致。

4. Nestemp 模板

xiebyl

Nestoc 的默认模板。

4.1. 参考文献管理

```
#nestemp.add-bib(read(encoding:none, "<xxx.bib>"))
```

TODO

4.2. 秘密等级管理

```
#show: nestemp.set-secret-level.with(n:<secret-level>)
```

秘密等级分为 4 级, 由不同颜色的页边标识:

0. 红■: 仅项目管理者、核心参与者可见
1. 黄■: 仅项目管理者、核心参与者、普通参与者可见
2. 绿■: 仅项目管理者、核心参与者、普通参与者、企业内部所有人可见
3. 白 : 所有人可见

秘密等级的规则：

- 标记了的秘密等级的文档：
 - `secret-level(父文档) <= secret-level(子文档)`
 - 即父文档的秘密等级不能弱于子文档的秘密等级
- 未标记秘密等级的文档：不受限制

一些例子：

父文档	子文档	
绿■	绿■	✓
绿■	白	✓
白	绿■	✗
绿■	无	✓
无	绿■	✓

表 1 秘密等级规则的例子

4.3. 术语管理

`#show: nestemp.init-glossary.with(<terms>)`

其中 terms 的用法参考 <https://github.com/swaits-typst-packages/glossy>。

5. 例子

本文档提供了一个具体的 Nestoc 例子，包含了 4 个模块：

- 总文档：doc/main.typ
- 父模块：doc/parent/main.typ
- 子模块：doc/parent/child/main.typ
- 孙模块：doc/parent/child/grandchild/main.typ

这 4 个模块：

- 每个模块均定义了自己的 `nestoc_fn` 函数。
- 每个模块均调用了 `nestoc`(自己的 `nestoc_fn`)。这让该模块可以编译成独立的 pdf 文档。
- 文档的嵌套通过调用 `nestoc`(别的模块的 `nestoc_fn`, `heading_offset: xx`)来实现。

所有 `nestoc` 和 `nestoc_fn` 的关系如下图所示：

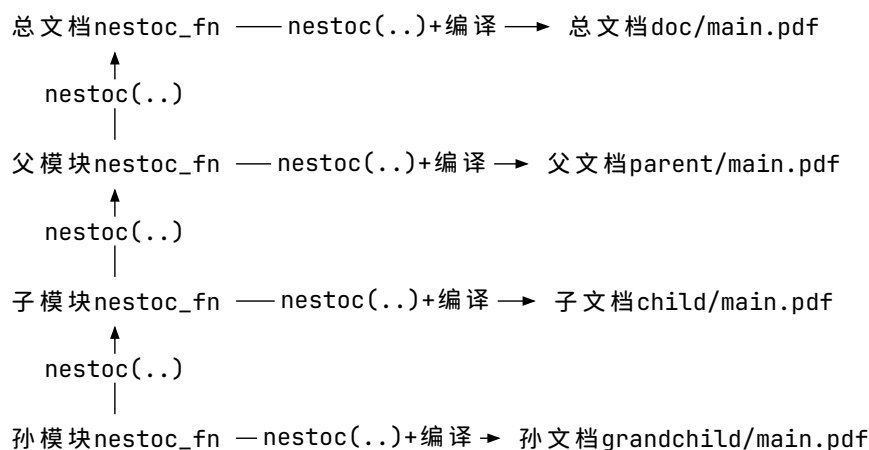


图 2 测试一下图

grandchild 被嵌套到了 child 里；child 被嵌套到了 parent 里；parent 被嵌套到了这个总文档的下面。你可以仔细观察这些 pdf 文档，然后理解它们间的嵌套关系。

5.1. 父模块的题目

xieby1

父模块 [1] 被嵌套到了总文档中，以二级标题为基础。即各级标题偏移+2。其各级标题映射关系如下：

父模块	总文档
题目	二级标题
一级标题	三级标题
..	..

表 2 父模块题目/标题映射关系

5.1.1. 父模块的一级标题

父模块的一级标题下方的内容：父模块术语○（Parent Term 0）。

第二次使用父模块术语○。

5.1.2. 子模块的题目

xieby1

子模块 [2] 被嵌入到父模块的题目之下。因此相对父模块，子模块的标题等级都会+1，这个现象能在父模块文档中观察到。当然，子模块间接地随着父模块被嵌套到了总文档中。相对总文档，子模块的标题登记都会+3 于是有映射关系如下：

子模块	父模块	总文档
题目	一级标题	三级标题
一级标题	二级标题	四级标题
..	..	..

表 3 子模块题目/标题映射关系

5.1.2.1. 子模块的一级标题

子模块的一级标题下方的内容：子模块术语○（Child Term 0）

第二次使用子模块术语○。

5.1.2.2. 孙模块的题目

xieby3

孙模块 [3] 被嵌套到子模块中，当然间接地随着子模块被嵌套到了父模块和总文档中。其各级标题的映射关系如下：

孙模块	子模块	父模块	总文档
题目	一级标题	二级标题	四级标题
一级标题	二级标题	三级标题	五级标题
..	..	..	..

表 4 孙模块题目/标题映射关系

5.1.2.2.1. 孙模块的一级标题

孙模块的一级标题下方的内容：孙模块术语○（Grandchild Term 0）。

第二次使用孙模块术语○。

6. 其他：文档注释（Doccom）

这个算是一个方法学，通过文档注释（Documentation + Comment => Doccom）的方式，将文档和代码紧密结合在一起。主要应用在底层文档，即需要和代码紧密结合的文档。

6.1. 要解决的问题

Doccom 要解决文档和代码分开而产生的问题：文档缺失、文档脱节、文档管理混乱。

6.2. 实现方法

实现方案很简单：

- 按照特定格式在注释里写 typst/markdown 文档，
- 按格式文档抽取出来，
- 保存为独立的 typst/markdown 文档即可。

6.3. 比较

- Doccom：简单，支持任意编程语言，任意文档语言；
- Doxygen：支持多种编程语言，但是极其复杂；
- Python/Java docstring, rustdoc, typst tidy：简单，但仅支持特定语言；

6.4. 例子

小节 3 和 小节 4 均采用了 Doccom 模式。以 小节 3 为例，其采用的 Doccom 格式为：

```
/*typ*/  
Here is the typst doc.  
...  
*/
```

只需要用正则表达式将 "Here is the typst doc..." 的内容抽取出来即可。小节 3 所涉及的 lib.typ 的源码如下：

```
1 /*typ*/  
2 #import "/lib.typ": nestoc, nestemp; #let nestoc_fn(heading_offset:0) =  
(  
3   title: "API", title-label: <chap:api>,  
4   author: "xieby1",  
5   abstract: [  
6     Nestoc 的所有功能围绕着函数  
7     ``typ #nestoc(nestoc_fn, heading_offset, template0, templateN) => body``  
8     展开。  
9   ],  
10  body: [  
11  
12 // TODO: API check  
13 #import "./nestemp/lib.typ" as nestemp  
14  
15 /*typ*/  
16 = ``typc nestoc(nestoc_fn, heading_offset:0, template0:nestemp.template0,  
17 templateN:nestemp.templateN) => body``  
18 对 `nestoc_fn` 包含的文档内容施加 `heading_offset` 的偏移。  
19 然后根据 `heading_offset` 施加样式：若 `heading_offset==0` 则应用样式 `template0`，否则应用样式 `templateN`。  
20  
21 - / 参数 `nestoc_fn`：为函数，见 @chap:nestoc_fn  
22 - / 参数 `heading_offset`：为 int，表示给 `nestoc_fn` 的题目和标题施加
```

```

`heading_offset`的偏移。
23  例如：
24  - / ````typc heading_offset:0````: 表示不偏移
25  - / ````typc heading_offset:1````: 表示`nestoc_fn`题目=>一级标题, `nestoc_fn`的
    一级标题=>二级标题, ...
26  - / ````typc heading_offset:2````: 表示`nestoc_fn`题目=>二级标题, `nestoc_fn`的
    一级标题=>三级标题, ...
27  - / 参数````typc template0````: 为函数, 见@chap:template0
28  - / 参数````typc templateN````: 为函数, 见@chap:templateN
29  - / 返回值: 为文档内容
30  */
31  #let nestoc(nestoc_fn, heading_offset: 0, template0: nestemp.template0, templateN:
    nestemp.templateN) = {
32    /*typ*//*
33      // TODO: remove indentation
34      = ````typc nestoc_fn(heading_offset:0) => nestemp_args```` <chap:nestoc_fn>
35
36      - / 参数`heading_offset`: 同`nestoc`的`heading_offset`
37      - / 返回值`nestemp_args`: `dict`类型, 有如下成员变量:
38        - / `title`: `content`类型, 文档题目
39        - / `title-label`: `label`类型, 文档题目的label
40          - 为什么需要`title-label`, 而不直接`title: [content <label>]`?
41            因为后者会报错"occurs multiple times in the document",
42            报错原因: heading在outline和body中各出现了一次。
43            此外后者的没有给title/heading打上label, 而是给content打上的label。
44        - / `author`: `str`类型, 作者
45        - / `abstract`: `content`类型, 摘要
46        - / `body`: `content`类型, 文档内容
47    */
48    let nestemp_args = nestoc_fn(heading_offset: heading_offset)
49    let body = nestemp_args.remove("body")
50    if heading_offset == 0 {
51      /*typ*//*
52        = ````typc template0(title:"", title-label:none, author:"", abstract:[],
        body) => styled_body```` <chap:template0>
53        - / 参数: 同`nestemp_args`
54        - / 返回值: 应用了模板之后的文档内容
55      */
56      template0(..nestemp_args, body)
57    } else {
58      /*typ*//*
59        = ````typc templateN(heading_offset, title:"", author:"", abstract:[], body)
        => styled_body```` <chap:templateN>
60        - 比`templateN`多了参数`heading_offset`。其他参数含义一致。
61      */
62      templateN(heading_offset, ..nestemp_args, body)
63    }
64  }
65
66  /*typ*//* ]); #nestoc(nestoc_fn) */
67

```

术语索引

子模块术语○. Child Term 0	5
孙模块术语○. Grandchild Term 0	5
父模块术语○. Parent Term 0	5

图索引

图 1 Nestoc 文档代码框架的图示	2
图 2 测试一下图	4

表格索引

表 1 秘密等级规则的例子	4
表 2 父模块题目/标题映射关系	5
表 3 子模块题目/标题映射关系	5
表 4 孙模块题目/标题映射关系	5

代码块索引

代码 1 Nestoc 文档的代码框架	2
---------------------------	---

参考文献

- [1] xieby1, 《Parent Bib Entry 0》. 2026 年.
- [2] xieby1, 《Child Bib Entry 0》. 2026 年.
- [3] xieby1, 《Grandchild Bib Entry 0》. 2026 年.